

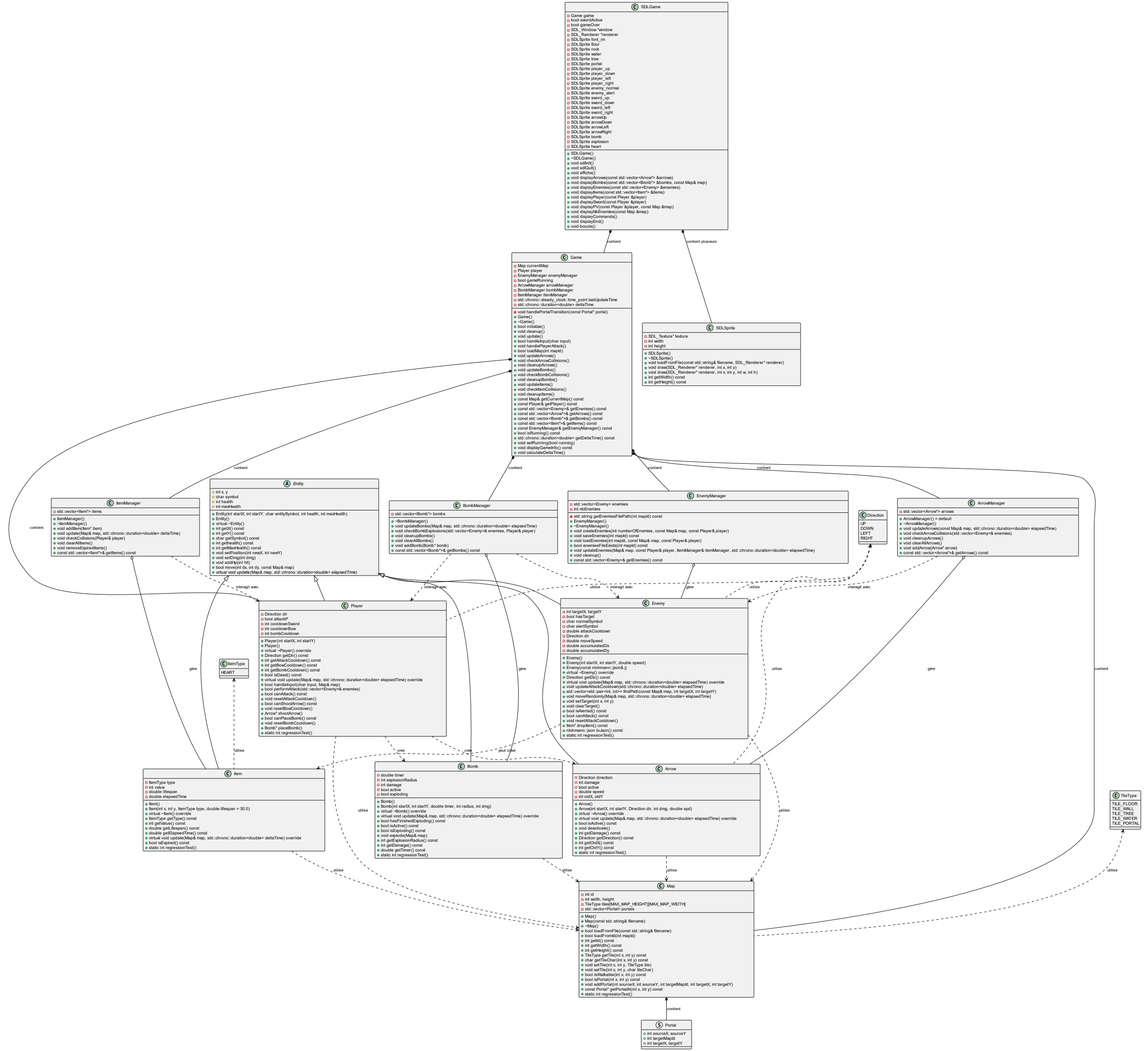


PROJET HK

HONG UGO P2307653
KOEUT ETHAN P2207925



Jeu de type RPG consistant en plusieurs cartes remplies de monstres.
L'objectif du joueur est d'abattre les 28 monstres du monde.



MAP

- Classe Map : Cœur de l'environnement de jeu
- Rôle central : Représente l'environnement de jeu avec une grille 2D de tuiles
- Gestion des tuiles :

Différents types (sol, mur, eau, arbre, portail)

Vérification de collision (isWalkable())

- Système de portails :

Connexion entre différentes cartes

Transition fluide entre les zones de jeu

- Chargement dynamique

Chargement depuis fichiers texte (loadFromFile())

Identification par ID (loadFromId())

- Interaction avec entités :

Fournit le contexte spatial pour tous les mouvements

Utilisée par toutes les entités pour la navigation

```
2
40 20
#####PP#####
#WWWWWWWW.....T..T.....#
#WWWWWWWWWW.....T..T.....#####
#WWWWWWWWWW.....T..T.....#####
#WWWWWWWWWW.....T..T.....#####
#WWWWWWWW.....T..T.....#####
#WWWWWW.....T..T.....#####
#TTTTTTTTTTTTT...TT..TT...TTTTTTTTTTTTT#
#.....#
#.....#
#.....#
#.....#
#TTTTTT...TTTTTT...TTTTTT...TTTTTT#
#.....T..T.....#
#...WWW...WWW...T..T.WWWWWW...WWW.#
#...WWWWWWWWWW...T..T.W.....W.#
#...WWWWWWWWWW...T..T.W.....W.#
#...WWW...WWW...T..T.WWWWWW...WWW.#
#.....T..T.....#
#####PP#####
4
19 19 1 19 1
20 19 1 20 1
19 0 3 19 18
20 0 3 20 18
```

ENTITY

- Classe Entity : Base abstraite du système d'objets
- Classe abstraite fondamentale :

Parent de tous les objets interactifs du jeu

- Propriétés communes essentielles :

Position (x, y) dans la grille de jeu

Symbole de représentation visuelle

Système de points de vie (health, maxHealth)

- Gestion des mouvements :

Méthode move() avec vérification de collision via Map

Positionnement contrôlé (setPosition())

- Système de santé universel :

Mécanismes de dégâts (setDmg())

Récupération de santé (addHp())

- Polymorphisme par méthode abstraite :

update() : comportement spécifique à chaque type d'entité

Permet une gestion unifiée des objets de jeu

- Hiérarchie : Player, Enemy, Arrow, Bomb et Item héritent tous de cette classe

A Entity

◇ int x, y
◇ char symbol
◇ int health
◇ int maxHealth

● Entity(int startX, int startY, char entitySymbol, int health, int maxHealth)
● Entity()
● virtual ~Entity()
● int getX() const
● int getY() const
● char getSymbol() const
● int gethealth() const
● int getMaxHealth() const
● void setPosition(int newX, int newY)
● void setDmg(int dmg)
● void addHp(int hlt)
● bool move(int dx, int dy, const Map& map)
● virtual void update(Map& map, std::chrono::duration<double> elapsedTime)

PLAYER

- Classe Player : Protagoniste contrôlable du jeu
- Entité centrale contrôlée par l'utilisateur :

Hérite d'Entity avec des fonctionnalités spécifiques au joueur

Point focal de l'expérience de jeu

- Système de direction et mouvement :

Orientation dans 4 directions (haut, bas, gauche, droite)

Gestion des entrées utilisateur via handleInput()

- Arsenal d'attaques diversifié :

Attaque au corps à corps avec épée (performAttack())

Tir de flèches à distance (shootArrow())

Placement de bombes stratégiques (placeBomb())

- Système de cooldown sophistiqué :

Gestion des délais entre attaques pour chaque arme

Méthodes de vérification (canAttack(), canShootArrow(), canPlaceBomb())

- Interaction avec l'environnement :

Détection des portails et transition entre cartes

Collecte d'objets

C Player
❑ Direction dir ❑ bool attackP ❑ int cooldownSword ❑ int cooldownBow ❑ int bombCooldown
● Player(int startX, int startY) ● Player() ● virtual ~Player() override ● Direction getDir() const ● int getAttackCooldown() const ● int getBowCooldown() const ● int getBombCooldown() const ● bool isDead() const ● virtual void update(Map& map, std::chrono::duration<double> elapsedTime) override ● bool handleInput(char input, Map& map) ● bool performAttack(std::vector<Enemy>& enemies) ● bool canAttack() const ● void resetAttackCooldown() ● bool canShootArrow() const ● void resetBowCooldown() ● Arrow* shootArrow() ● bool canPlaceBomb() const ● void resetBombCooldown() ● Bomb* placeBomb() ● static int regressionTest()

ENEMY

- Classe Enemy : Adversaires intelligents et dynamiques
- Antagonistes adaptatifs :

Hérite d'Entity avec comportement hostile spécifique

Différents états (normal/alerte) avec symboles distincts

- IA de mouvement évoluée :

Déplacement aléatoire en mode normal (moveRandomly())

Pathfinding vers le joueur en mode alerte (findPath())

Vitesse de mouvement paramétrable

- Système de détection et poursuite :

Acquisition de cible (setTarget()) quand joueur à proximité

Changement d'état visuel lors de l'alerte

- Mécanisme d'attaque temporisé :

Cooldown entre attaques (updateAttackCooldown())

Dégâts au joueur lors du contact

- Système de récompense :

Possibilité de lâcher des objets à la mort (dropItem())

- Persistance : Sauvegarde/chargement via JSON pour préserver l'état entre

C Enemy
❑ int targetX, targetY ❑ bool hasTarget ❑ char normalSymbol ❑ char alertSymbol ❑ double attackCooldown ❑ Direction dir ❑ double moveSpeed ❑ double accumulatedDx ❑ double accumulatedDy
● Enemy() ● Enemy(int startX, int startY, double speed) ● Enemy(const nlohmann::json& j) ● virtual ~Enemy() override ● Direction getDir() const ● virtual void update(Map& map, std::chrono::duration<double> elapsedTime) override ● void updateAttackCooldown(std::chrono::duration<double> elapsedTime) ● std::vector<std::pair<int, int>> findPath(const Map& map, int targetX, int targetY) ● void moveRandomly(Map& map, std::chrono::duration<double> elapsedTime) ● void setTarget(int x, int y) ● void clearTarget() ● bool isAlerted() const ● bool canAttack() const ● void resetAttackCooldown() ● Item* dropItem() const ● nlohmann::json toJson() const ● static int regressionTest()

ENEMY MANAGER

- Classe EnemyManager : Orchestration des adversaires
- Gestion centralisée des ennemis :

Conteneur et contrôleur de tous les ennemis du jeu

Simplifie l'interface entre le jeu et les instances d'ennemis

- Création et distribution stratégique :

Génération d'ennemis avec paramètres adaptés (createEnemies())

Placement intelligent évitant le joueur et respectant la carte

- Persistance des données :

Sauvegarde de l'état des ennemis par carte (saveEnemies())

Chargement contextuel selon la carte active (loadEnemies())

Format JSON pour flexibilité et maintenabilité

- Mise à jour collective optimisée :

Actualisation synchronisée de tous les ennemis (updateEnemies())

Gestion des interactions avec joueur et environnement

- Cycle de vie des objets :

Création d'items lors de l'élimination d'ennemis

Nettoyage des ressources lors des transitions

C Enemy
❑ int targetX, targetY ❑ bool hasTarget ❑ char normalSymbol ❑ char alertSymbol ❑ double attackCooldown ❑ Direction dir ❑ double moveSpeed ❑ double accumulatedDx ❑ double accumulatedDy
● Enemy() ● Enemy(int startX, int startY, double speed) ● Enemy(const nlohmann::json& j) ● virtual ~Enemy() override ● Direction getDir() const ● virtual void update(Map& map, std::chrono::duration<double> elapsedTime) override ● void updateAttackCooldown(std::chrono::duration<double> elapsedTime) ● std::vector<std::pair<int, int>> findPath(const Map& map, int targetX, int targetY) ● void moveRandomly(Map& map, std::chrono::duration<double> elapsedTime) ● void setTarget(int x, int y) ● void clearTarget() ● bool isAlerted() const ● bool canAttack() const ● void resetAttackCooldown() ● Item* dropItem() const ● nlohmann::json toJson() const ● static int regressionTest()

GAME

- Classe Game : Moteur central du jeu
- Coordinateur principal :

Intègre tous les sous-systèmes (joueur, ennemis, objets, carte)

Gère le flux global et l'état du jeu

- Boucle de jeu complète :

Méthode update() synchronisant toutes les entités

Gestion du temps avec deltaTime pour animations fluides

Détection des collisions et interactions entre entités

- Gestion des entrées utilisateur :

Traitement des commandes via handleInput()

Déclenchement des actions appropriées (mouvement, attaque)

- Système de transition entre cartes :

Chargement dynamique des cartes (loadMap())

Gestion des portails et téléportation du joueur

- Orchestration des managers spécialisés :

Délégation aux gestionnaires (EnemyManager, ArrowManager, etc.)

Coordination des interactions entre systèmes

- Séparation des responsabilités : Isole la logique de jeu de l'interface graphique, permettant les modes texte et graphique avec le même moteur de jeu

C Game
❑ Map currentMap ❑ Player player ❑ EnemyManager enemyManager ❑ bool gameRunning ❑ ArrowManager arrowManager ❑ BombManager bombManager ❑ ItemManager itemManager ❑ std::chrono::steady_clock::time_point lastUpdateTime ❑ std::chrono::duration<double> deltaTime
■ void handlePortalTransition(const Portal* portal) ● Game() ● ~Game() ● bool initialize() ● void cleanup() ● void update() ● bool handleInput(char input) ● void handlePlayerAttack() ● bool loadMap(int mapId) ● void updateArrows() ● void checkArrowCollisions() ● void cleanupArrows() ● void updateBombs() ● void checkBombCollisions() ● void cleanupBombs() ● void updateItems() ● void checkItemCollisions() ● void cleanupItems() ● const Map& getCurrentMap() const ● const Player& getPlayer() const ● const std::vector<Enemy>& getEnemies() const ● const std::vector<Arrow*>& getArrows() const ● const std::vector<Bomb*>& getBombs() const ● const std::vector<Item*>& getItems() const ● const EnemyManager& getEnemyManager() const ● bool isRunning() const ● std::chrono::duration<double> getDeltaTime() const ● void setRunning(bool running) ● void displayGameInfo() const ● void calculateDeltaTime()

ARROW

- Classe Arrow : Projectiles dynamiques
- Projectile avec trajectoire directionnelle :

Hérite d'Entity avec comportement spécifique aux projectiles

Déplacement linéaire selon une direction fixe

- Propriétés de combat :

Valeur de dégâts configurable

Système d'état actif/inactif pour gestion du cycle de vie

- Physique de mouvement :

Vitesse paramétrable affectant la distance parcourue

Conservation des coordonnées précédentes pour détection de collision

- Interaction avec l'environnement :

Désactivation automatique lors de collision avec murs

Détection de collision avec ennemis via ArrowManager

- Mise à jour basée sur le temps :

Déplacement progressif basé sur elapsedTime

Mouvement par étapes pour vérification précise des collisions

C Arrow
▣ Direction direction ▣ int damage ▣ bool active ▣ double speed ▣ int oldX, oldY
● Arrow() ● Arrow(int startX, int startY, Direction dir, int dmg, double spd) ● virtual ~Arrow() override ● virtual void update(Map& map, std::chrono::duration<double> elapsedTime) override ● bool isActive() const ● void deactivate() ● int getDamage() const ● Direction getDirection() const ● int getOldX() const ● int getOldY() const ● static int regressionTest()

BOMB

- Classe Bomb : Explosifs stratégiques
- Arme à zone d'effet temporisée :

Hérite d'Entity avec mécanisme d'explosion différée

Placement stratégique pour piéger ou débloquer des zones

- Système de minuterie :

Compte à rebours configurable avant détonation

Suivi du temps via elapsedTime et detonationTime

- Explosion avec rayon d'action :

Dégâts dans un rayon configurable (explosionRadius)

Affecte à la fois les ennemis et potentiellement le joueur

- États visuels distincts :

Symbole normal ('O') avant explosion

Symbole d'explosion ('*') pendant la détonation

- Gestion d'état post-explosion :

Méthode hasJustExploded() pour synchroniser effets visuels

Système de nettoyage automatique via BombManager

C Bomb

```
❑ double timer
❑ int explosionRadius
❑ int damage
❑ bool active
❑ bool exploding

● Bomb()
● Bomb(int startX, int startY, double timer, int radius, int dmg)
● virtual ~Bomb() override
● virtual void update(Map& map, std::chrono::duration<double> elapsedTime) override
● bool hasFinishedExploding() const
● bool isActive() const
● bool isExploding() const
● void explode(Map& map)
● int getExplosionRadius() const
● int getDamage() const
● double getTimer() const
● static int regressionTest()
```

ITEM

- Éléments collectables diversifiés :

Hérite d'Entity avec propriétés spécifiques aux objets

Types variés (cœur, etc.) avec effets distincts

- Système de valeur et effet :

Valeur numérique déterminant la puissance de l'effet

Application automatique lors de la collecte par le joueur

- Cycle de vie temporel :

Durée de vie limitée configurable (lifespan)

Disparition automatique après expiration (isExpired())

- Représentation visuelle distinctive :

Symboles spécifiques selon le type (ex: 'H' pour cœur)

Facilite l'identification visuelle par le joueur

- Intégration avec l'économie du jeu :

Apparition suite à l'élimination d'ennemis

Gestion centralisée via ItemManager pour distribution équilibrée

Item

```
❑ ItemType type
❑ int value
❑ double lifespan
❑ double elapsedTime
```

```
● Item()
● Item(int x, int y, ItemType type, double lifespan = 30.0)
● virtual ~Item() override
● ItemType getType() const
● int getValue() const
● double getLifespan() const
● double getElapsedTime() const
● virtual void update(Map& map, std::chrono::duration<double> deltaTime) override
● bool isExpired() const
● static int regressionTest()
```


SDL GAME

- Classe SDLGame : Interface graphique SDL2
- Moteur de rendu graphique :

Implémentation visuelle du jeu utilisant SDL2

Transformation de la logique abstraite en éléments visuels

- Boucle de jeu graphique :

Méthode boucle() gérant événements et rendu

Synchronisation temporelle pour animations fluides (60 FPS)

- Gestion des entrées utilisateur :

Capture des événements clavier via SDL_Event

Mappage vers les commandes du jeu (handleInput())

- Système de rendu :

Affichage de sprites pour entités et environnement

Effets visuels pour actions (attaques, explosions)

Interface utilisateur avec informations de jeu

SDLGame

- Game game
- bool swordActive
- bool gameOver
- SDL_Window *window
- SDL_Renderer *renderer
- SDL_Sprite font_im
- SDL_Sprite floor
- SDL_Sprite rock
- SDL_Sprite water
- SDL_Sprite tree
- SDL_Sprite portal
- SDL_Sprite player_up
- SDL_Sprite player_down
- SDL_Sprite player_left
- SDL_Sprite player_right
- SDL_Sprite enemy_normal
- SDL_Sprite enemy_alert
- SDL_Sprite sword_up
- SDL_Sprite sword_down
- SDL_Sprite sword_left
- SDL_Sprite sword_right
- SDL_Sprite arrowUp
- SDL_Sprite arrowDown
- SDL_Sprite arrowLeft
- SDL_Sprite arrowRight
- SDL_Sprite bomb
- SDL_Sprite explosion
- SDL_Sprite heart

- SDLGame()
- ~SDLGame()
- void sdlInit()
- void sdlQuit()
- void affiche()
- void displayArrows(const std::vector<Arrow*> &arrows)
- void displayBombs(const std::vector<Bomb*> &bombs, const Map& map)
- void displayEnemies(const std::vector<Enemy> &enemies)
- void displayItems(const std::vector<Item*> &items)
- void displayPlayer(const Player &player)
- void displaySword(const Player &player)
- void displayPV(const Player &player, const Map &map)
- void displayNbEnemies(const Map &map)
- void displayCommands()
- void displayEnd()
- void boucle()

CONCLUSION

Le diagramme de Gantt et le diagramme des classes sont différents entre le début et la fin du projet.

Certaines fonctionnalités n'ont pas été implémentés comme la sauvegarde, de l'audio ou la gestion d'un inventaire.

Nous avons eue des difficultés lors de la gestion des cooldowns, les déplacements des flèches, pour la sauvegarde des monstres en fichiers JSON, ou encore pour la création de sprites.